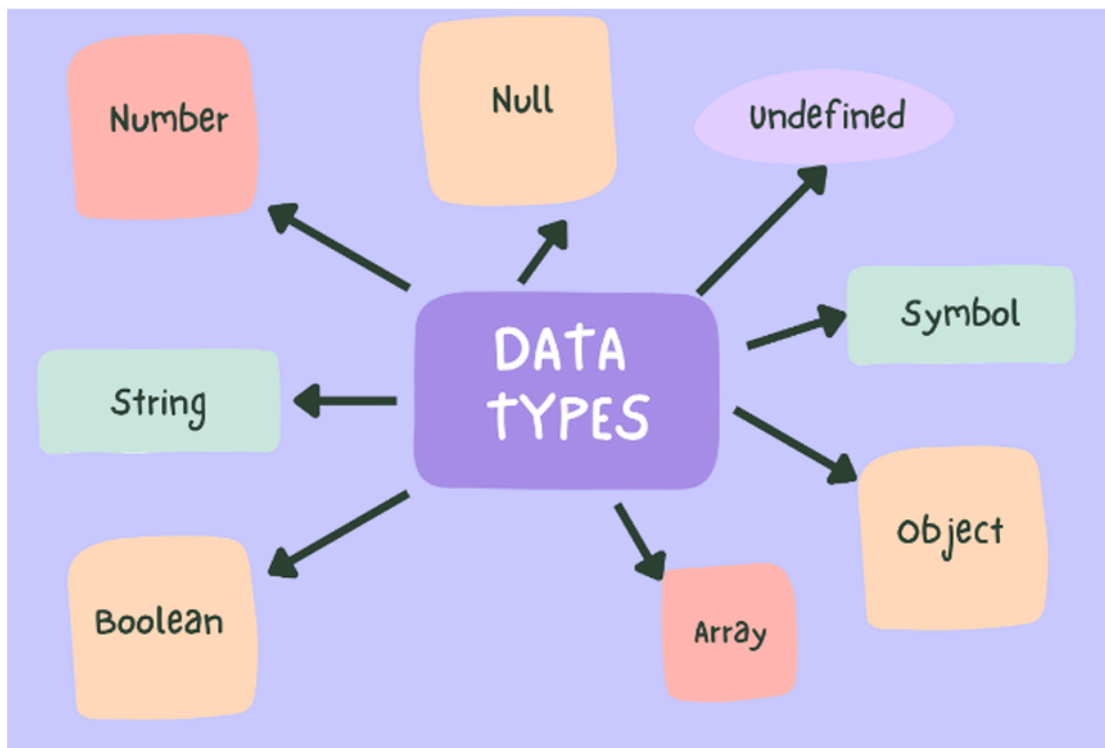


JS Datatypes

1. What is meant by Datatypes?

- a. Javascript provides different datatypes to hold different types of values.
- b. Datatypes define which kind or type of data a variable can hold or store.
- c. Javascript is a dynamic type language, meaning you don't need to specify the type of the variable because it is dynamically used by the JS engine.
- d. In javascript, datatypes are dynamic, meaning a variable can hold different types of data at different times.



2. Why do we need datatypes in JS?

a. Datatypes are essential for several reasons:

- i. **Memory Allocation** → Datatypes help the JavaScript engine determine how much memory to allocate for a variable.
- ii. **Code Clarity** → Knowing the datatype clarifies what kind of data a variable stores, improving code readability and maintainability.
- iii. **Operations** → Different datatypes support different operations. For example, mathematical operations work with numbers, while string operations are specific to text data.

3. Types of Datatype

a. JavaScript has two main categories of datatypes:

- i. Primitive Datatype
- ii. Non-Primitive Datatype

4. Primitive Datatypes

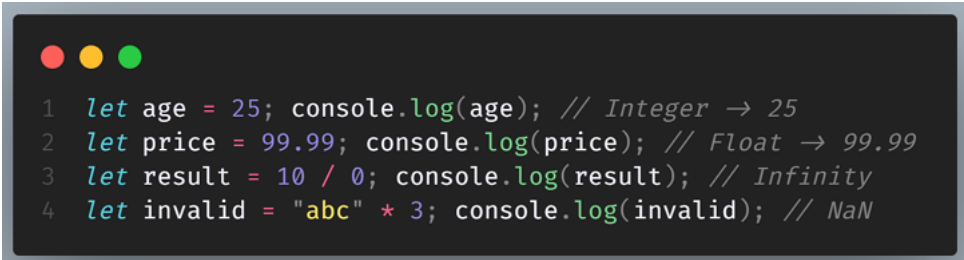
a. Primitive data types are the basic built-in types provided by JavaScript.

b. There are 7 primitive data types in JavaScript:

- i. Number
- ii. String
- iii. Boolean
- iv. Undefined
- v. Null
- vi. BigInt
- vii. Symbol

c. **Number** Datatype ⇒

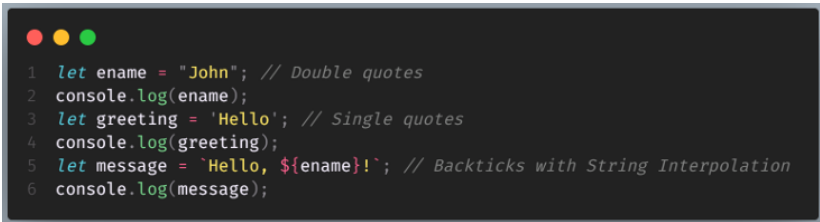
- i. It represents numeric values, including integers and floating-point numbers.
- ii. Special numeric values include:
 1. Infinity → (e.g., dividing by zero)
 2. Infinity → (negative infinity)
 3. NaN → Not-a-Number, (e.g., invalid arithmetic)
- iii. For Example →



```
1 let age = 25; console.log(age); // Integer → 25
2 let price = 99.99; console.log(price); // Float → 99.99
3 let result = 10 / 0; console.log(result); // Infinity
4 let invalid = "abc" * 3; console.log(invalid); // NaN
```

d. **String** Datatype ⇒

- i. It is used to store text data.
- ii. The string data type in JavaScript represents a sequence of characters that are surrounded by single or double quotes or template literals typically used to represent text.
- iii. Strings are enclosed in quotes:
 1. Single Quotes (' ')
 2. Double Quotes (" ")
 3. Backticks or String Template Literals (` `)
- iv. For Example →



```
1 let ename = "John"; // Double quotes
2 console.log(ename);
3 let greeting = 'Hello'; // Single quotes
4 console.log(greeting);
5 let message = `Hello, ${ename}!`; // Backticks with String Interpolation
6 console.log(message);
```

e. **Boolean** Datatype ⇒

- i. JavaScript Boolean represents *true* or *false* values.
- ii. It's used for logical operations, condition testing, and variable assignments based on conditions.
- iii. For Example →

```
1 let isLoggedIn = true;
2 let hasAccess = false;
3 console.log(5 > 3); // true (means 5 is greater than 3)
4 console.log(10 < 5); // false (means 5 is not greater than 10)
```

f. **Undefined** Datatype ⇒

- i. This means that a variable has been declared but has not been assigned a value, or it has been explicitly set to the value *undefined*.
- ii. For Example →

```
1 let notAssigned;
2 console.log(notAssigned); //undefined
```

g. **Null** Datatype ⇒

- i. In JavaScript, null indicates the deliberate (intentional) absence of any object value.
- ii. For Example →

```
1 let emptyValue = null;
2 console.log(emptyValue); // null
```

h. **BigInt** Datatype ⇒

- i. Used to represent very large integers beyond the safe limit of the **Number** type ($2^{53} - 1$).
- ii. A BigInt value, also known as a bigint primitive, is created by appending **n** to an integer literal.
- iii. For Example →

```
1 var a=4521123545123244n;  
2 console.log(a); // 4521123545123244n
```

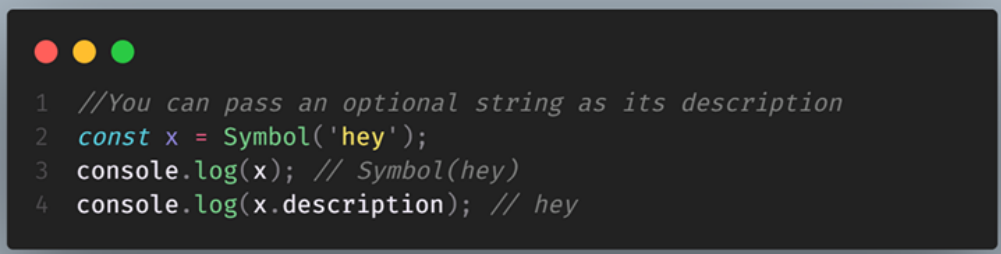
i. **Symbol** Datatype ⇒

- i. The JavaScript ES6 introduced a new primitive datatype called **Symbol**.
- ii. Symbols are immutable (cannot be changed) and are unique.
- iii. In JavaScript, symbol is one of the primitive types and can be created using the **Symbol()** factory method that *returns a different symbol each time*.
- iv. For Example →

```
1 // two symbols with the same description  
2 const value1 = Symbol('hello');  
3 const value2 = Symbol('hello');  
4  
5 console.log(value1 === value2); // false
```

- v. Though value1 and value2 both contain the same description, they are different.
- vi. **Access Symbol Description** → To access the *description* of a symbol, we use the **.** operator.

vii. For Example →



```
1 //You can pass an optional string as its description
2 const x = Symbol('hey');
3 console.log(x); // Symbol(hey)
4 console.log(x.description); // hey
```

5. Non-Primitive Datatypes:

- a. Non-primitive data types, also known as reference types, are objects and derived data types.
- b. They can store collections of values or more complex entities.
- c. They are mutable.

d. Types of non-primitive data types in JavaScript:

- i. Object → Represents a collection of key-value pairs.
- ii. Array → Represents an ordered list of values.
- iii. Function → Functions in JavaScript are also objects, allowing them to be assigned to variables, passed as arguments, and returned from other functions
- iv. Date → An object used for handling dates and times.
- v. Map → A collection of key-pair data items, where the keys can be of any type.
- vi. Set → A collection of unique values.
- vii. WeakMap → A collection of key-value pairs where the keys are objects and the values can be of any type, with weak references to keys.
- viii. WeakSet → A collection of unique objects, with weak references to those objects.

6. Difference Between Primitive and Non-Primitive Datatypes:

- a. In JavaScript, datatypes are classified as either primitive or non-primitive. Primitive types are fundamental and immutable, while non-primitive types are more complex and mutable.

Primitive Data Types	Non-Primitive Data Types
Immutable (Cannot be changed)	Mutable (Can be changed)
Stored directly in the variable	Stored by reference
number, string, boolean, null, undefined, symbol	object, array
Checked by value	Checked by reference
Fixed size	Dynamic size
Simple operations (copy by value)	Complex operations (copy by reference)
Used for simple data values	Used for more complex structures and collections

7. The **typeof** Operator:

- a. The `typeof` operator can be used to find out what type of data a variable or operand contains.
- b. It is a keyword use as an unary operator to identify the type of data stored.
- c. `typeof` will check the value belongs to which category.
- d. Syntax:

```
let num = 10;  
console.log(typeof(num)); // Number
```